

ASP.NET TUTORIALS

INTRODUCTION TO ASP.NET CORE MVC

What is .NET:

.NET is a free, cross-platform, open source developer platform for building many different types of applications. With .NET, you can use multiple languages, editors, and libraries to build for web, mobile, desktop, games, IOT, and more.

What is ASP.NET:

ASP.NET is a free web framework for building great websites and web applications using HTML, CSS, and JavaScript. You can also create Web APIs and use real-time technologies like Web Sockets. ASP.NET Core is an alternative to ASP.NET

OR

ASP.NET extends the .NET developer platform with tools and libraries specifically for building web apps.

Introduction to ASP.Net Core:

- Nowadays, everyone is talking about open source, cross platform development. Microsoft has always been known for its Windows based products, but now we are in the new age of development. For this, a new revolutionary product came into the market, which is Microsoft .NET Core.
- NET Core is a free open source and cross platform framework, created for building modern Cloud based Applications and every .NET developer feels proud of it. Now, there are no boundaries for the platform. Now, every .NET developer can say, yes I am platform independent, I am using an open source.
- In the revolution of software development, Microsoft launched its first .NET framework in the year 2000 with its first .NET framework 1.0. This

framework plays a major role in the field of software development. People love Microsoft technology products, because these products are easy to use and easy to learn.

- In the year 2016, Microsoft has come up with a new revolution, Microsoft .NET Core 1.0. People always love ASP.NET, because it's working over WorldWideWeb.
- ASP.NET Core is a new open source, cross platform framework to create modern Web based Cloud based systems, which means, now you are not only working for Windows, you can run in Linux, Mac; i.e., wherever you want.

OPEN SOURCE MEANS:

- In general, open source refers to any program whose source code is made available for use or modification as users or other developers see fit. Open source software is usually developed as a public collaboration and made freely available.
- The software being distributed must be redistributed to anyone else without any restriction.
- The source code must be made available (so that the receiving party will be able to improve or modify it).

ASP.NET Core Features:

- Cross platform, open source now runs your app over Linux, Windows, Mac; i.e., wherever you want.
- Fast Development- fast work over the Browsers.
- Work in your editors - now you can work not only in Visual Studio. You can also choose Visual Studio code.

.NET CORE Features:

- ASP.NET Core MVC
- ASP.Net Core API
- IOT
- Universal Windows App.

Difference:

<u>.NET FRAMEWORK</u>	<u>.NET CORE FRAMEWORK</u>
OLD Framework	New Framework
Not Open Source	Open Source
ONLY FOR WINDOWS PLATFORM	CROSS PLATFORM (WINDOWS, LINUX, MAC)
<u>ASP.NET</u>	<u>ASP.NET CORE</u>
Targets .Net framework	Targets .Net core framework
All components are available like asp.Net MVC, web API, asp.Net web Forms etc.	Asp.Net web forms are not there in asp.Net core, we have asp.Net MVC core and web API in .Net core.

History of ASP.NET:

- ASP.NET has been used from many years to develop web applications. Since then, the framework went through a steady evolutionary change and finally led us to its most recent descendant ASP.NET Core 1.0.
- ASP.NET Core 1.0 is not a continuation of ASP.NET 4.6.

- It is a whole new framework, a side-by-side project which happily lives alongside everything else we know.
- It is an actual re-write of the current ASP.NET 4.6 framework, but much smaller and a lot more modular.
- Some people think that many things remain the same, but this is not entirely true. ASP.NET Core 1.0 is a big fundamental change to the ASP.NET landscape.

History of ASP.NET Core:

- ASP.NET Core is an open source and cloud-optimized web framework for developing modern web applications that can be developed and run on Windows, Linux and the Mac. It includes the MVC framework, which now combines the features of MVC and Web API into a single web programming framework.
- ASP.NET Core apps can run on .NET Core or on the full .NET Framework.
- It consists of modular components with minimal overhead, so you retain flexibility while constructing your solutions.

Pre-Requisite:

- HTML
- CSS
- JavaScript, JQuery
- Bootstrap
- C# Programming language.
- MVC
- Visual Studio 2017 or higher version

- We use Visual Studio 2019 in this tutorial series.

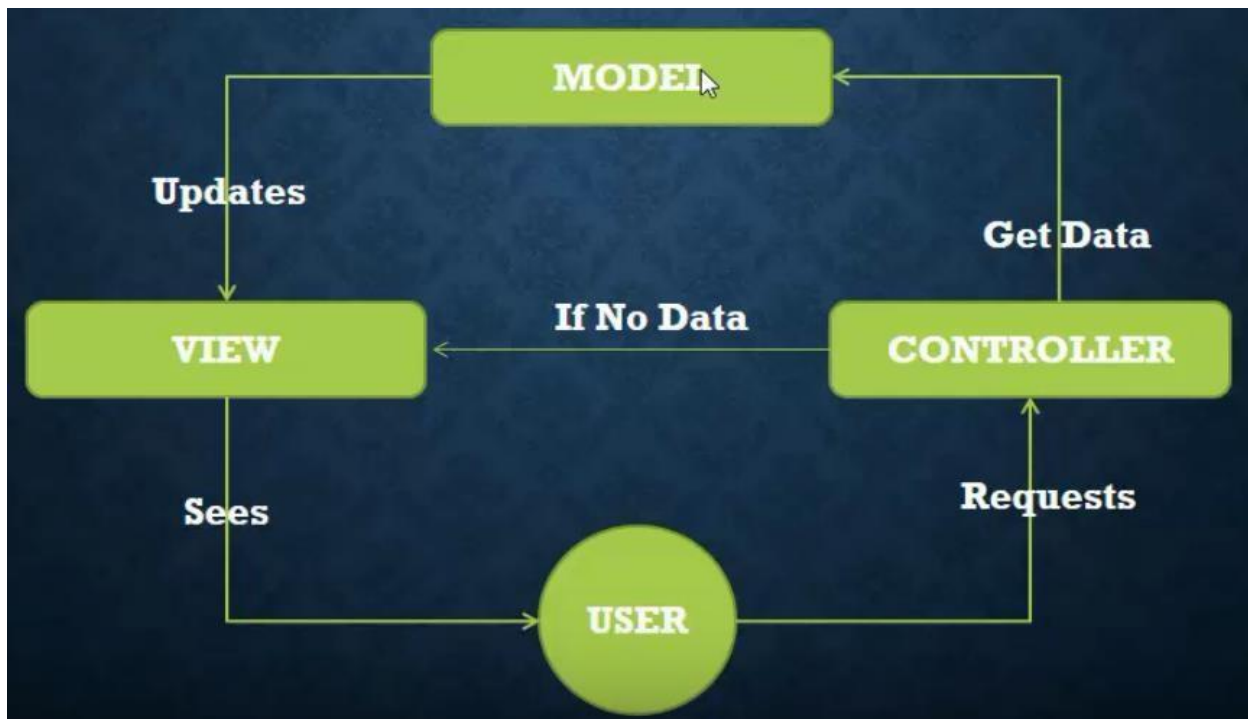
What is MVC Pattern:

- **MVC** Stand for **Model - View – Controller**
- **MVC** is an architectural design pattern. It means this design pattern is used at the architecture level of an application.
- Model, View & Controller are the main components of MVC pattern.

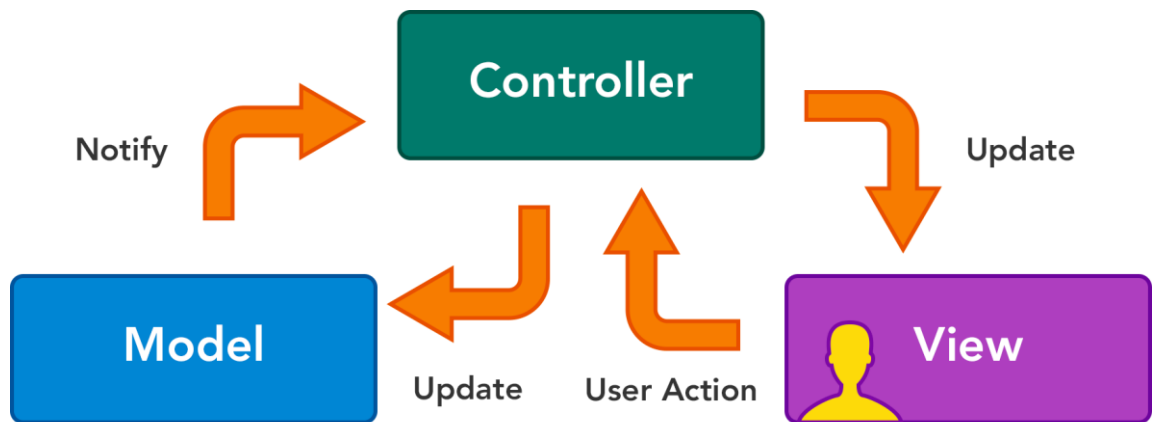
Role of components of MVC:

- Model (Business Entities)
- View (Presentation Logic)
- Controller (Business Logic)

Work Flow of MVC:



MVC WORKFLOW:



Real Life Example:



Controller:

- A controller is .cs (for c#) file which have some methods called Action Method.
- When a request comes on controller, it actually hits an action method.
- Represents the logic responsible for coordination between the view and model classes.
- Now everything depends on Action method what to return from it. It may return only view, only data, or both of them.

Model:

- A model in Asp.Net Core MVC is a simple class which has some properties.
- Handles requests to the application by the user.
- Model is used to pass data from controller (action method) to view and vice versa i.e View to Controller (action method).
- Model is also used for server side data validation. And with some technique we can generate client side validation also.
- It is not mandatory that each action method will return some model.

View:

- A View is a combination of Html and C#.
- Hence for C# application the extension of a view is. CSHTML.
- Whatever you see on your browser is a view.
- When a view (containing C#) gets rendered on browser then all its C# is converted into HTML. It means on a browser we will only see HTML and data.

Benefits of MVC:

- Separation of concern.
- Each component has a specific job hence it is easy to debug the code.

Advantage of Separation of Concerns(SOS):

- Allows work on individual pieces of the system in isolation
- Facilitates reusability
- Ensures the maintainability of a system
- Ensures extensibility
- Enables users to better understand the system

Environment Set in Machine (window, Linux, MacOS):

Editor or IDE (Integrated Development Environment).

- Visual Studio (Community)
- Visual Studio Code
- Sublime etc.

Dot Net Core SDK:

Download and Install dot Net core SDK. (any version).

Source: <https://dotnet.microsoft.com/en-us/download>

Check is it Install?

Command: `dotnet --version`

CREATE Project ASP.NET CORE MVC

System Configuration:

OS: Windows, Linux , MacOS

RAM: At least 8 GB

Hard Drive must be: SSD.

Hard Drive Must have 50GB Space

Environment Set in Machine (window, Linux, MacOS):

Editor or IDE (Integrated Development Environment).

- Visual Studio (Community)
- Visual Studio Code
- Sublime etc.

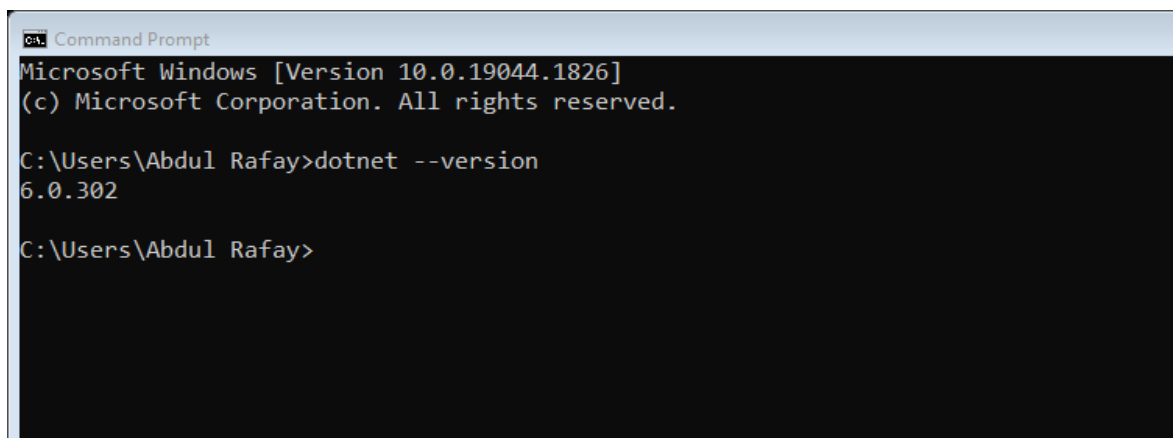
Dot Net Core SDK:

Download and Install dot Net core SDK. (any version).

Source: <https://dotnet.microsoft.com/en-us/download>

Check is it Install is complete?

Command: dotnet --version



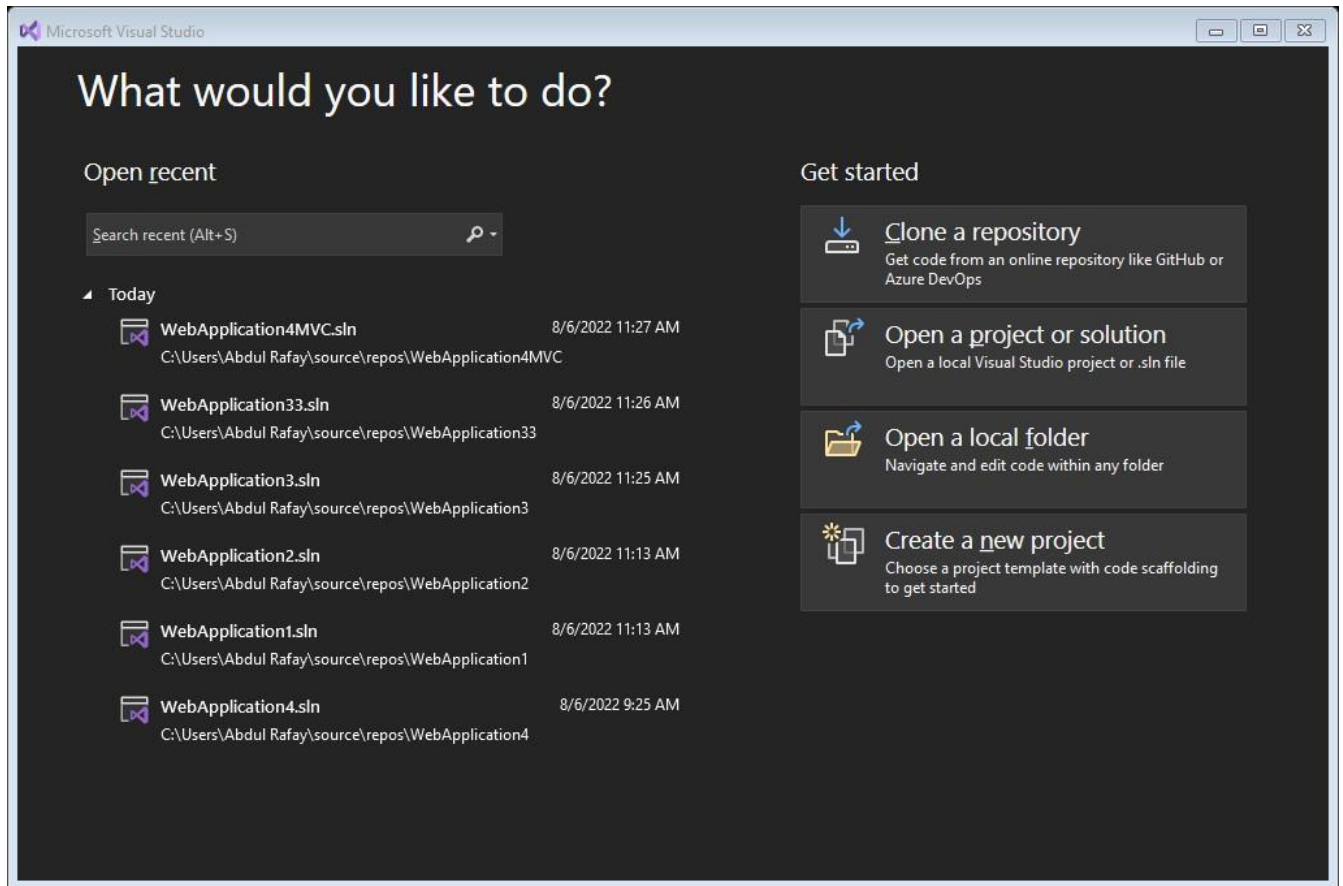
```
Command Prompt
Microsoft Windows [Version 10.0.19044.1826]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Abdul Rafay>dotnet --version
6.0.302

C:\Users\Abdul Rafay>
```

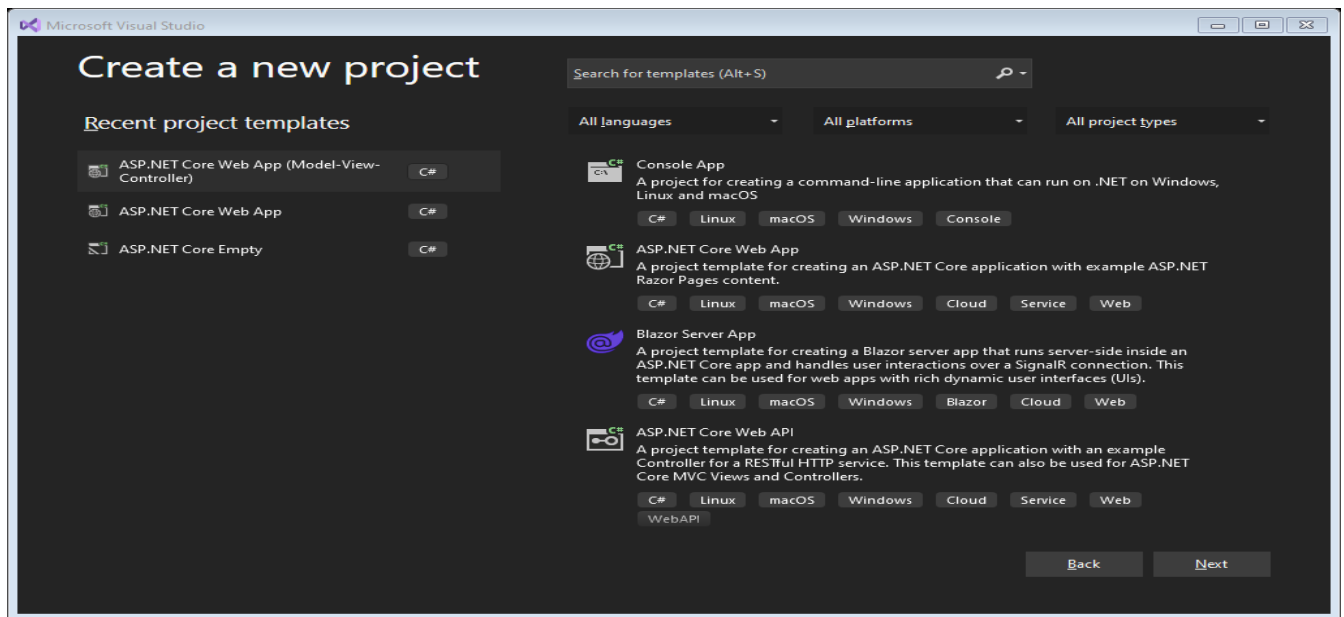
Start Visual Studio:

Create a New Project.



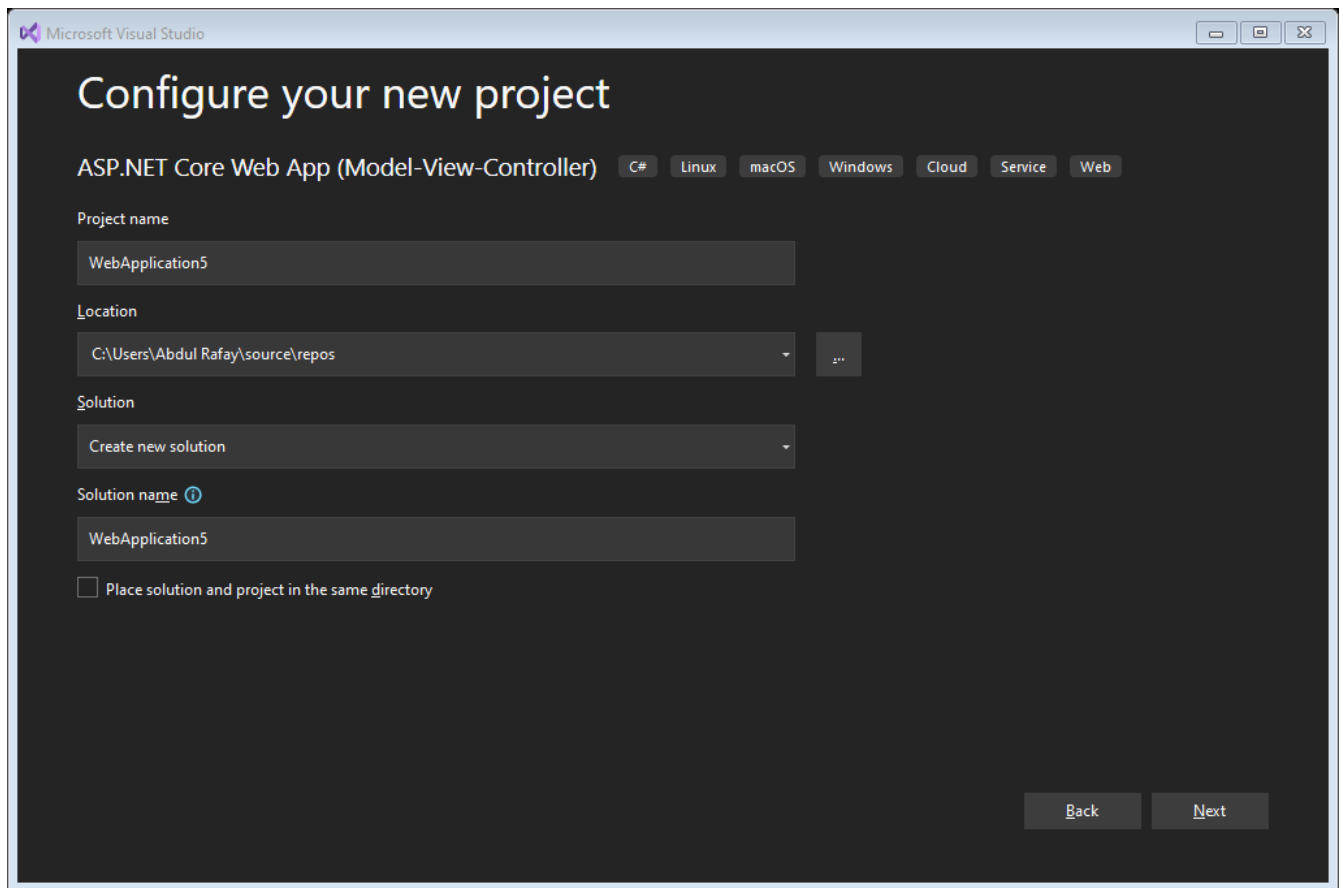
Select Technology with Template:

We will select ASP.NET Core Web Application(MVC).

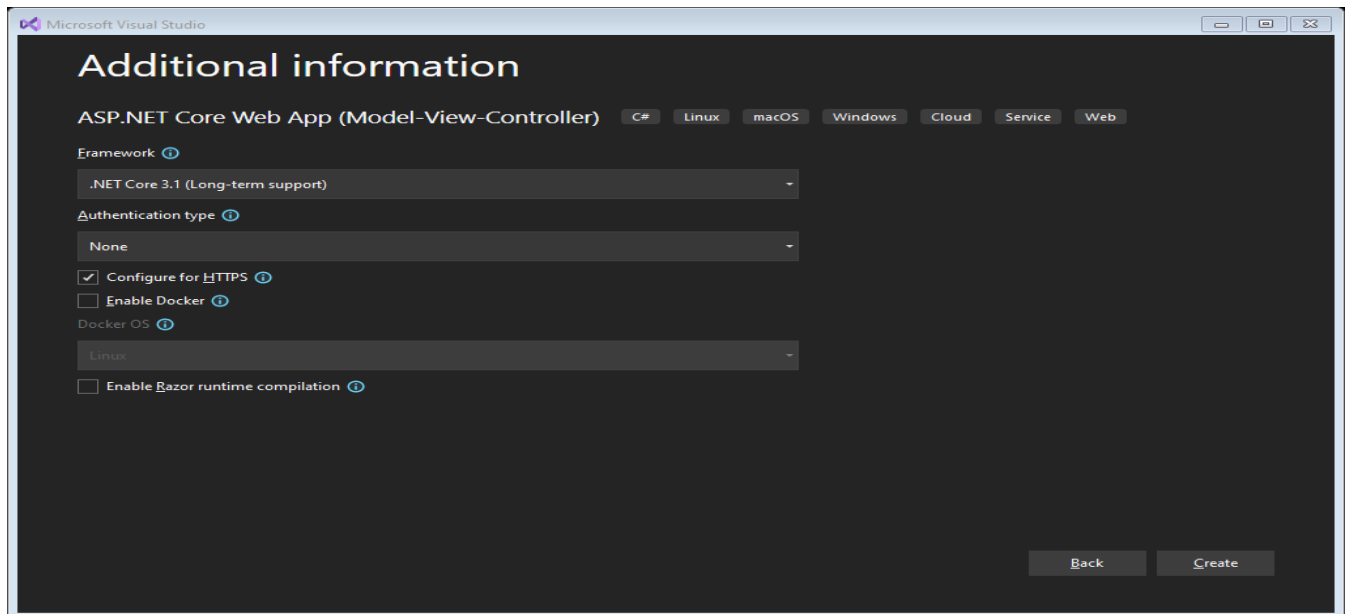


Configure the Project:

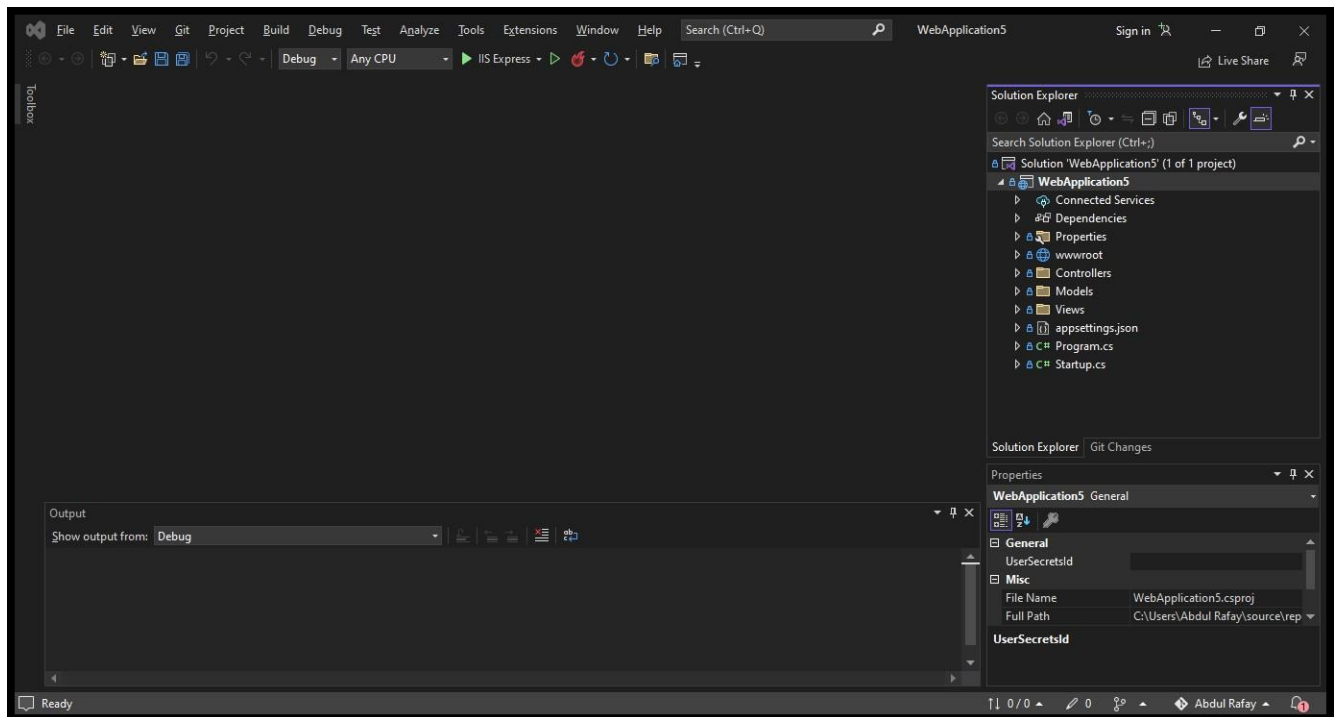
- Set Name
- Set Directory



Additional Information(Select .Net Core version):



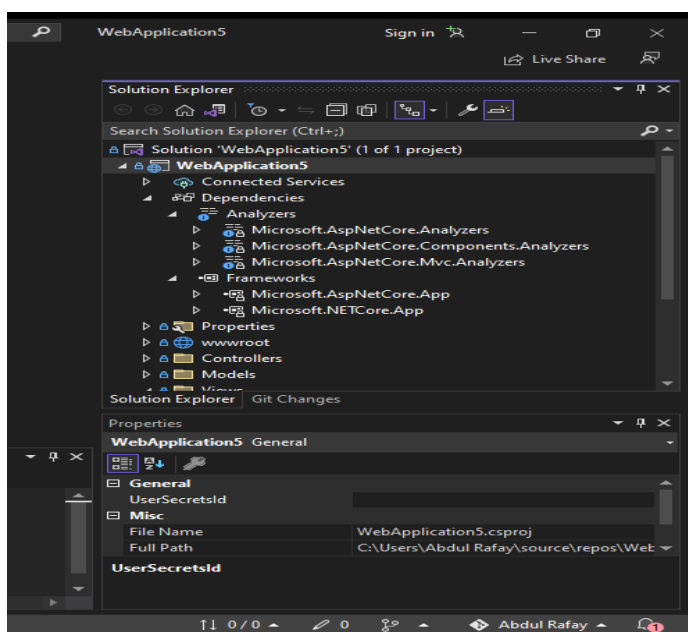
Complete build Project:



ASP.NET Core:

Here, you will learn about the project structure and significance of each file created by ASP.NET Core 2.0 application template using Visual Studio 2017.

The following is a default project structure when you create an empty ASP.NET Core application in Visual Studio.



Dependencies:

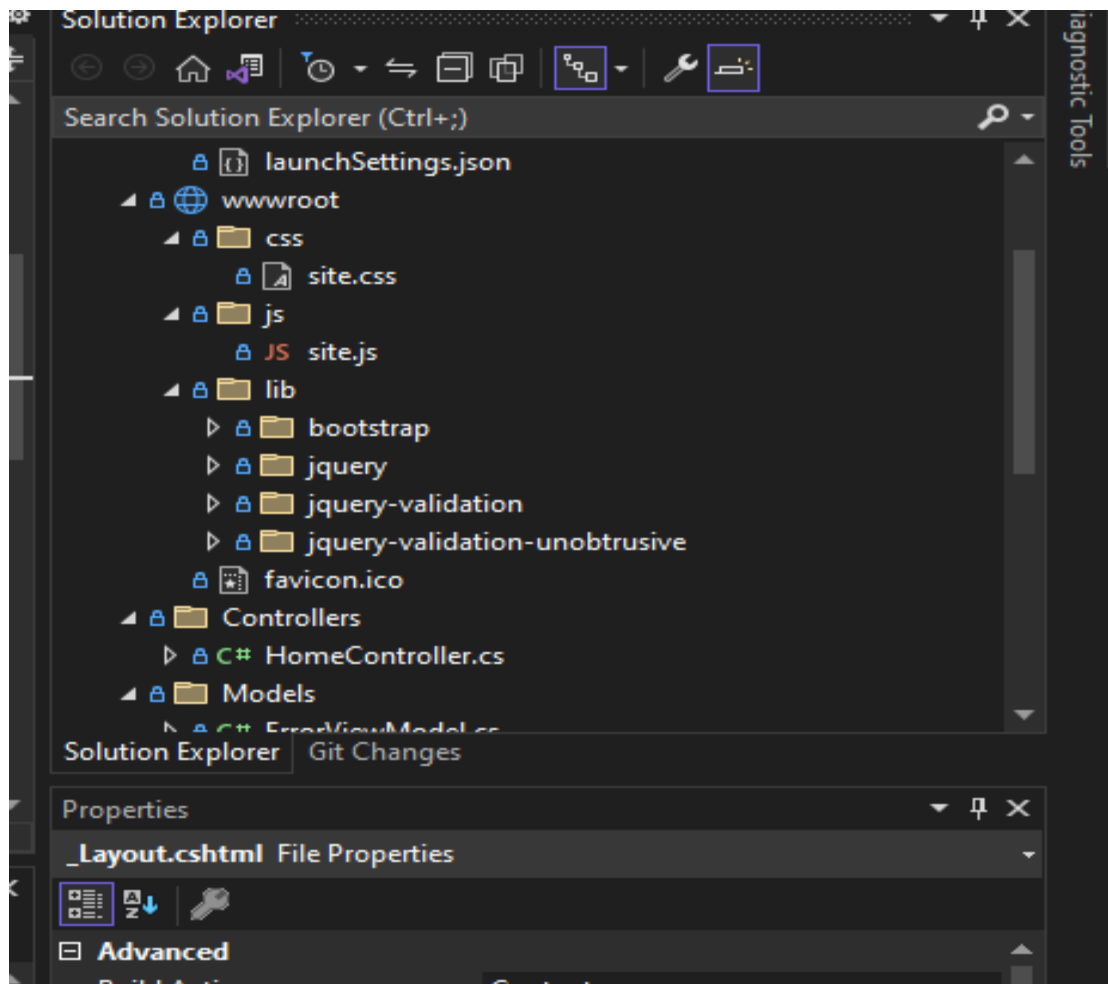
The Dependencies in the ASP.NET Core 3.1 project contain all the installed server-side NuGet packages, as shown below.

ASP.NET Core – WWWRoot Folder:

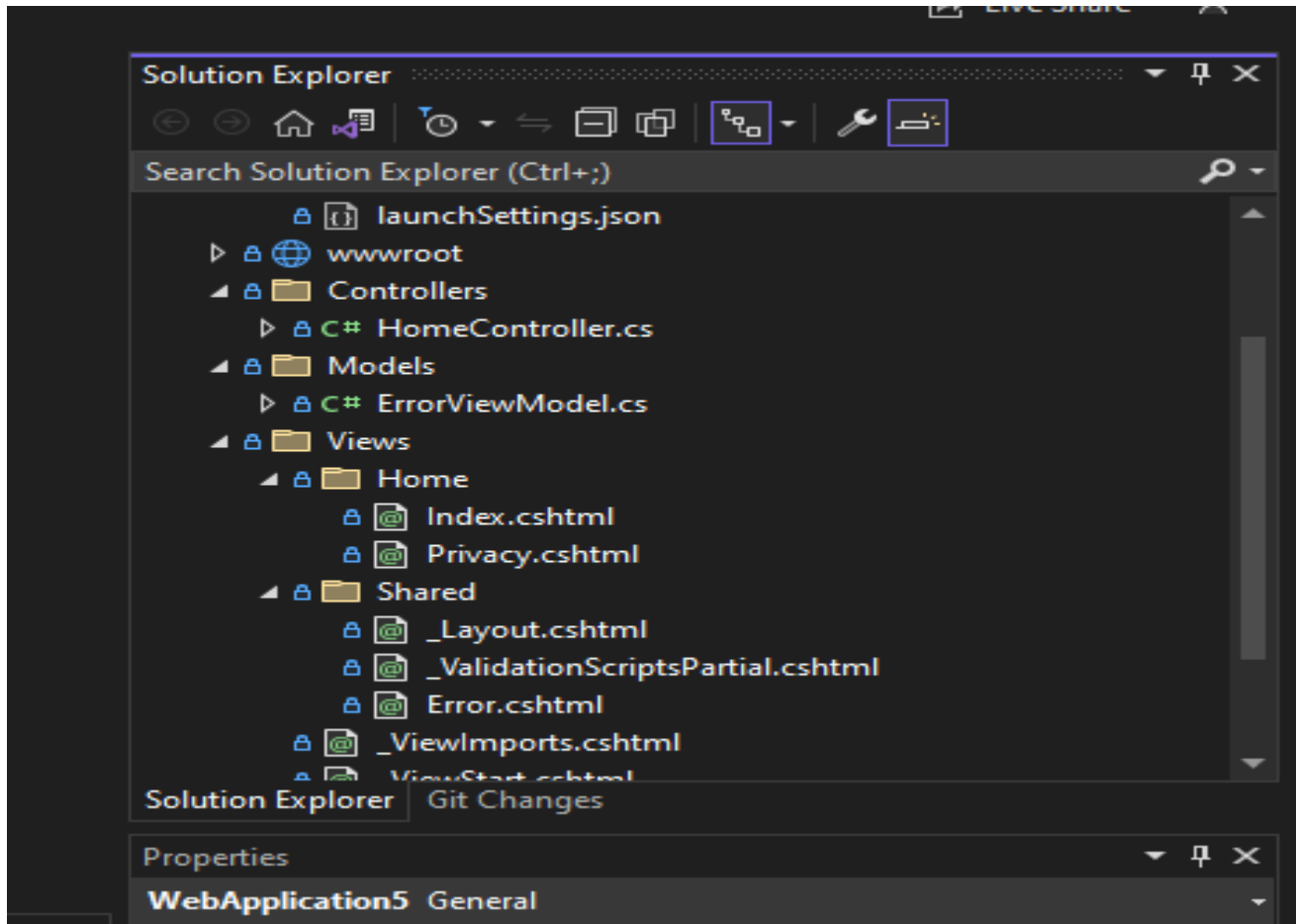
By default, the **wwwroot** folder in the ASP.NET Core project is treated as a web root folder. Static files can be stored in any folder under the web root and accessed with a relative path to that root.

In the standard ASP.NET application, static files can be served from the root folder of an application or any other folder under it. This has been changed in ASP.NET Core. Now, only those files that are in the web root - wwwroot folder can be served over an http request. All other files are blocked and cannot be served by default.

Generally, there should be separate folders for the different types of static files such as JavaScript, CSS, Images, library scripts etc. in the wwwroot folder as shown below.



Project MVC (Model View Controller) Structure:



Controller:

The Controllers folder contains class files for the controllers. A Controller handles users' request and returns a response. MVC requires the name of all controller files to end with "Controller". You will learn about the controller in the next section.

Model:

The Models folder contains model class files. Typically model class includes public properties, which will be used by the application to hold and manipulate application data.

View:

The Views folder contains HTML files for the application. Typically view file is a .cshtml file where you write HTML and C# or VB.NET code.

The Views folder includes a separate folder for each controller. For example, all the .cshtml files, which will be rendered by HomeController will be in View > Home folder.

The Shared folder under the View folder contains all the views shared among different controllers e.g., layout files.

MVC IN DETAIL WITH PRACTICAL

What is Controller?

A controller, in an ASP.NET CORE MVC application does the following:

- A Controller is a special class with .cs (C#) extension.
- Manages the flow of the application.
- Is responsible for intercepting incoming requests and executing the appropriate application code.
- Communicates with the models of the application and selects the required view to be rendered for the request.
- In MVC template a Controller class is inherited from Controller class.
- Allows separating the business logic of the application from the presentation logic.

In an ASP.NET MVC application, a controller is responsible to:

- Locate the appropriate method to call for an incoming request.
- Validate the data of the incoming request before invoking the requested method.
- Retrieve the request data and passing it to requested method as arguments.
- Handle any exceptions that the requested method throws.
- Help in rendering the view based on the result of the requested method.
- Caching, Security etc. can also be applied on controllers.

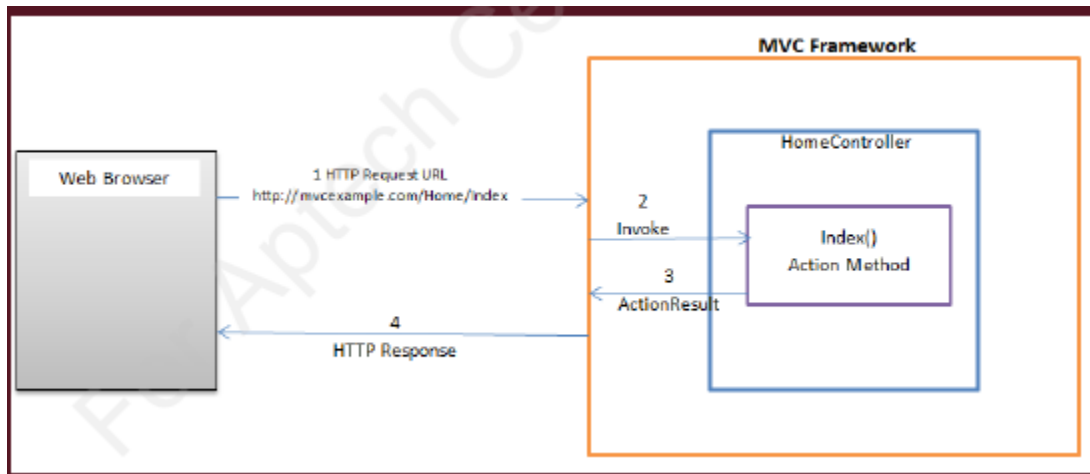
Action Method in Controller

- All the public methods of the Controller class are called Action methods. They are like any other normal methods with the following restrictions
- All Method in Controller called Action Method.

Restriction on Action Method:

1. Action method must be public. It cannot be private or protected
2. Action method cannot be overloaded
3. Action method cannot be a static method.

Let's Practical of Controller.



What is View:

- A view is a file .cshtml (C sharp HTML) extension.
- A view is the combination of programming language & HTML.
- View are generally return form action method.

Role of View:

- A View in asp.net core MVC application is responsible for UI i.e application data presentation.
- We display information About the website on browser using Views.
- A user generally performs all the actions on a view.
- Location of Views in **View folder**.

What is View Engine?

- The View Engine is responsible for producing an HTML response when invoked by the Controller Action method.
- View Engine is a piece of code which is used to render server side code into view.

The Controller Action methods can return various types of responses, which are collectively called as Action Results. The ViewResult is the ActionResult which produces the HTML Response.

What is Razor View Engine?

The Razor View Engine is the default View Engine for the ASP.NET Core apps. It looks for Razor markup in the View File and parses it and produces the HTML response.

Razor Syntax?

Everything starts with @

The following are the two ways; by which you can achieve the transitions.

- Using Razor code expressions
- Using Razor code blocks.

Practical:

Make Basic Application with some views.

- Conditional Statements
- Loops
- Use Static Files (images,css,js)
- Install Bootstrap, font-awesome other library

What is WWWROOT:

By default, the wwwroot folder in the ASP.NET Core project is treated as a **web root folder**. Static files can be stored in any folder under the web root and accessed with a relative path to that root.

Razor File Compilation:

Install : Microsoft.aspnetcore.mvc.razor.runtimecompilation

And add services in configure file.

```
services.AddRazorPages().AddRazorRuntimeCompilation();
```

- A model is the Class with .cs (C#) extension.

Responsible of Model:

- A model is responsive for data.
- We Get/Set data from/to a source in form of Model.
- The Model in an MVC application represents the state of the application and any business logic or operations that should be performed by it.
- Business logic should be encapsulated in the model, along with any implementation logic for persisting the state of the application.
- Strongly-typed views typically use ViewModel types designed to contain the data to display on that view. The controller creates and populates these ViewModel instances from the model.

MVC IN DETAIL WITH PRACTICAL

What is Model:

- A model is the Class with .cs (C#) extension.

Responsible of Model:

- A model is responsive for data.
- We Get/Set data from/to a source in form of Model.
- The Model in an MVC application represents the state of the application and any business logic or operations that should be performed by it.
- Business logic should be encapsulated in the model, along with any implementation logic for persisting the state of the application.
- Strongly-typed views typically use ViewModel types designed to contain the data to display on that view. The controller creates and populates these ViewModel instances from the model.

Make One Model Class:

Example:

```
namespace complete_web_application.Models
{
    public class student
    {
```

```

        public int stid{ get; set; }
        public string stname{ get; set; }

        public int stage{ get; set; }

    }
}

```

In start of Controller:

```

public List<student> getstudents()
{
    return new List<student>()
    {
        new student(){ stid = 1, stname="farhan" , stage = 20},
        new student(){ stid = 2, stname="ali" , stage =12},
        new student(){ stid = 3, stname="rehman" , stage = 27}
    };
}

```

Create Controller:

```

public List<student> getstudents()
{
    return new List<student>()
    {
        new student(){ stid = 1, stname="farhan" , stage = 20},
        new student(){ stid = 2, stname="ali" , stage =12},
        new student(){ stid = 3, stname="rehman" , stage = 27}
    };
}

```

Check URL:

<https://localhost:44346/home/getstudents>

Show model list data on View:

Action Method:

```

public IActionResult getstudents()
{
    var data = getstudents();
    return View(data);
}

```

View:

```
@model IEnumerable<complete_web_application.Models.student>;
```

```
<h1>getbooks</h1>
```

```
<div class="container">
```

```
<div class="row">
```

```
@foreach (var item in Model)
```

```
{
```

```
<div class="col-md-4">
```

```
<div class="card border px-5">
```

```
<p>@item.stid</p>
```

```
<p>@item.sname</p>
```

```
<p>@item.stage</p>
```

```
</div>
```

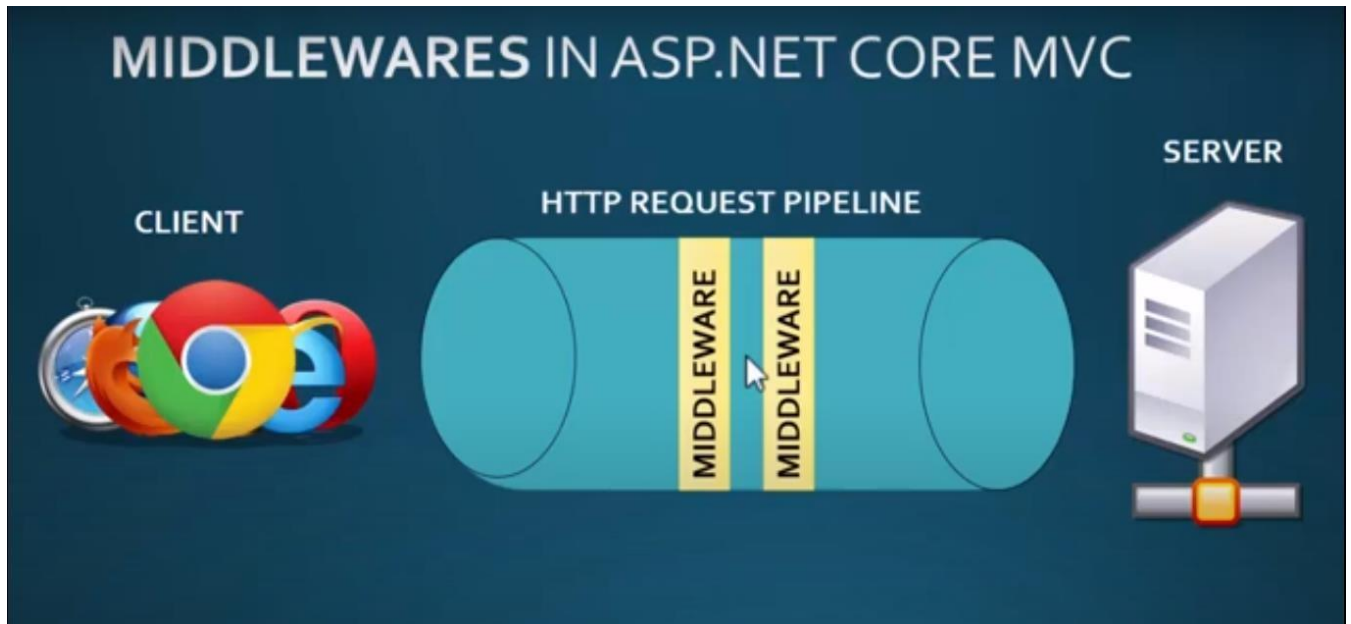
```
</div>
```

```
}
```

```
</div>
```

```
</div>
```

MIDDLEWARES IN ASP.NET CORE MVC



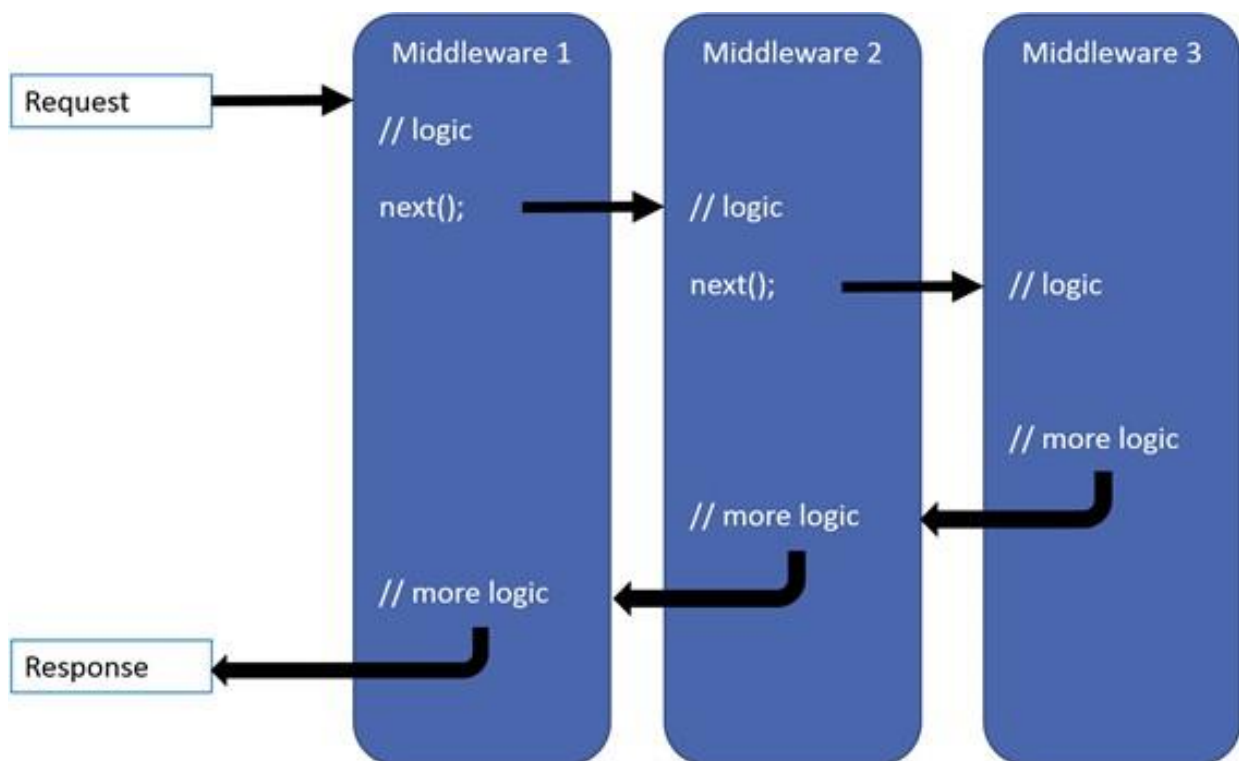
Introduction:

- ASP.NET Core introduced a new concept called Middleware. A middleware is nothing but a component which is executed on every request in ASP.NET Core application.
- Middleware in ASP.NET Core controls how our application responds to HTTP requests.
- Middleware are software components that are assembled into an application pipeline to handle request and responses.
- It can also control how our application looks when there is an error.
- It is a key piece in how we **authenticate** and **authorize** a user to perform specific actions.
- Each piece of middleware in ASP.NET Core is an object, and each piece has a very specific, focused, and limited role.
- Ultimately, we need many pieces of middleware for an application to behave appropriately.
- Middleware has access to all the request and response.

- Middleware is a series of components present in this processing pipeline.
- Middleware are located in **Configure** method in **Startup.cs** class file.
- We can manage multiple middleware's for some specific task like,
 - Middleware's for Routing – e-g: `app.UseMvcWithDefaultRoute();`
 - Middleware's for Static files – e-g: `app.UseStaticFiles();`
 - Middleware's for MVC functionality – e-g: `app.UseMvc();`

Configure Middleware:

- We can configure middleware in the **Configure** method of the Startup class using `IAApplicationBuilder` instance generally called `app`.



Basic Example:

```
app.Use(async (context, next) =>
```

```
{
    await context.Response.WriteAsync("This si farhan webstie");
});
```

Example:

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("hello this is middleware 1");
    await next();

    await context.Response.WriteAsync("hello this is middelware 1 resonse");

});

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("this is middleware 2");

    await next();

});

app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
```

For Work MVC:

```
app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapDefaultControllerRoute();
});
```

ROUTING MVC WITH PRACTICAL

There are two types of Routing in ASP.NET CORE MVC.

1. Convention based Routing (Default in MVC)
2. Attribute Routing

Attribute Routing example:

```
[Route("")]
[Route("Home")]
[Route("Home/Index")]
[Route("Home/Home")]
public IActionResult Index()
{
    return View();
}
[Route("Home/Home/Detail/{id}")]
public int Detail(int id)
{
    return id;
}
```

Razor Engine In ASP.NET CORE MVC:

Basics:

<h1>This is website : @DateTime.Now</h1>

<h1>@DateTime.Now.Year</h1>

@for (var i = 0; i < 10; i++)

{

```
<p>This is Counter: @i</p>
}
```

Another example:

```
@{
    var a = 10;
    var b = "Farhan";
}
```

```
<p>This is @a</p>
```

```
<p>This is @b</p>
```

Another:

```
@{
    string[] names = {"farhan", "ali", "afan", "shahid"};
    foreach(var item in names)
    {
        <h2>This is @item</h2>
    }
}
```

MVC IN DETAIL WITH PRACTICAL

What is ViewBag?

The ViewBag in ASP.NET MVC is used to transfer temporary data (which is not included in the model) from the controller to the view.

Internally, it is a dynamic type property of the ControllerBase class which is the base class of the Controller class.

This type of data binding is known as loosely binding.

Example:

```
ViewBag.mytitle = "Home Page by Farhan";
```

```
ViewBag.data = new { Id = 1, name = "Farhan" };
```

View:

```
<h1>@ViewBag.mytitle</h1>
```

```
<p>@ViewBag.data</p>
```

What is ViewData?

In ASP.NET MVC, ViewData is similar to ViewBag, which transfers data from Controller to View. ViewData is of [Dictionary](#) type, whereas ViewBag is of [dynamic type](#). However, both store data in the same dictionary internally.

ViewData is a dictionary, so it contains key-value pairs where each key must be a string.

Example:

```
ViewData["prop1"] = new { id = 1, name = "farhan jani " };
```

View:

```
var info = ViewData["prop1"];
```

```
<p>@info</p>
```

What is TempData?

In ASP.NET MVC, TempData is similar to ViewData, which transfers data from Controller to another View. TempData is of [Dictionary](#) type, whereas ViewData is of [dynamic type](#). However, both store data in the same dictionary internally.

TempData is a dictionary, so it contains key-value pairs where each key must be a string.

Example:

```
TempData["prop1"] = new { id = 1, name = "farhan " };
```

View:

```
var info = TempData["prop1"];  
<p>@info</p>
```

What is Strongly Types View?

- Strongly typed view or strong types object is used to pass data from controllers to view.
- The View which binds with any model is called as strongly types view.
- You can bind any class as a model to view.

Example:

```
public IActionResult Index()  
{  
    var student = new List<Student>  
    {  
        new Student{ rollid = 1, stname = "Fahran" , standard = "10th"},  
        new Student{ rollid = 2, stname = "ALI" , standard = "11th"},  
        new Student{ rollid = 3, stname = "AHMED" , standard = "12th"},  
    };  
  
    return View(student);  
}
```

```
@{
@model IEnumerable<tutorial2.Models.Student>
}

<table border="1">

    @{
        foreach (var item in students)
        {
            <tr>
                <td>@item.rollid</td>
                <td>@item.sname</td>
                <td>@item.standard</td>
            </tr>
        }
    }

</table>
```

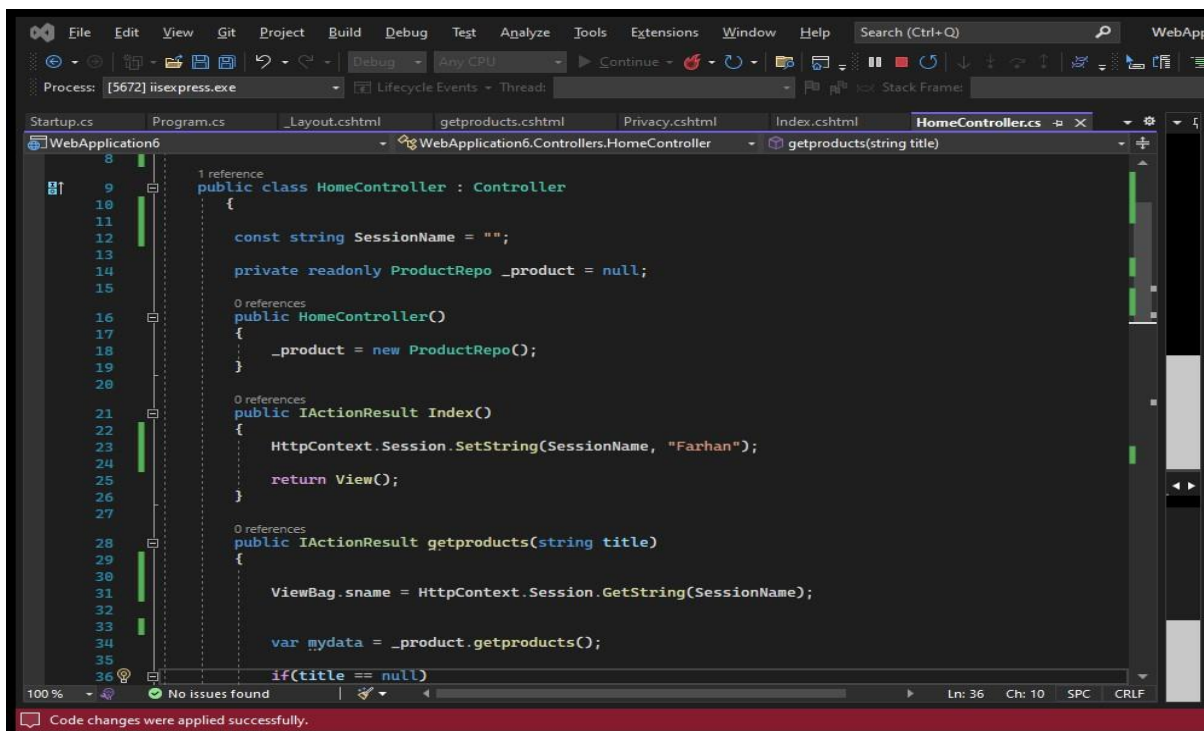
MVC IN DETAIL WITH PRACTICAL

What is Session?

Session state is an ASP.NET Core scenario for storage of user data while the user browses a web app. Session state uses a store maintained by the app to persist data across requests from a client. The session data is backed by a cache and considered ephemeral data.

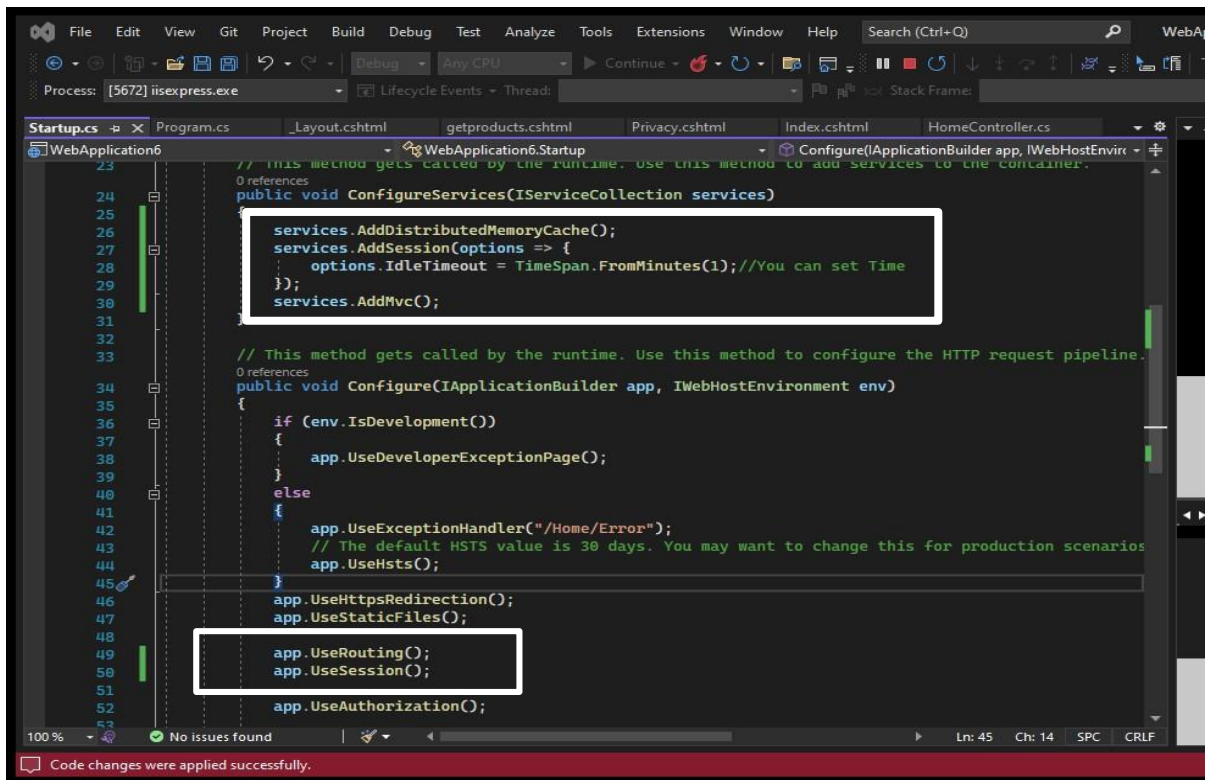
Steps of include Session in Application:

1. Download Session Package from “Manage NuGet Packages.
2. Work on Controller:



```
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626

```



MVC IN DETAIL WITH PRACTICAL

Html Helper Methods In Asp.Net Mvc 5:

In ASP.NET MVC Framework, helper methods:

- Are extension methods to the Html Helper class, can be called only from views.
- An HTML helper is a method that is used to render html content in a view.
- Simplifies the process of creating a view.
- Allows generating HTML markup that you can reuse across the Web application.

Some of the commonly used helper methods while developing an MVC application are as follows:

- `Html.ActionLink()`
- `Html.BeginForm()` and `Html.EndForm()`
- `Html.Label()`
- `Html.TextBox()`
- `Html.TextArea()`
- `Html.Password()`
- `Html.CheckBox()`
- `Html.Hidden()`

Html.ActionLink():

`Html.ActionLink()` helper method allows you to generate a hyperlink that points to an action method of a controller class.

The general syntax of the `Html.ActionLink()` helper method is as follows:

`@Html.ActionLink(<link_text>,<action_method>,<optional_controller>)`

An HTML helper is a method that is used to render html content in a view

- To create the html for a textbox with id = “fullName” and name = “fullName”
 - `<input type="text" name = “fullName” id = “fullName”>`

OR

- We can use the “TextBox” Html helper class method.
 - `@Html.TextBox(“fullName”)`
- There are several overloaded versions. To set a value, along with name of a textbox
 - `@Html.TextBox(“fullName”, “Adil”)`

To set HTML attributes, use the following overloaded version. Note that: We are passing HTML attributes (style and title) as an anonymous type.

```

14
15 @Html.ActionLink("Go to Privacy","Privacy","Home")
16 <br/>
17
18 @Html.TextBox("fullname", "Muhammad Farhan", new {
19     style = "background-color: red; color: green;",
20
21 })
22
23 @Html.TextBox("fullname", "Muhammad Farhan", new { @class = "form-control" })
24

```

- We can use external style sheet to give the style to the textbox.
- Applying bootstrap class to textbox.
- If you want to use attributes that are reserved in Csharp programming language like class, readonly etc. Then you have to use @ symbol before the name of attribute.

Html.BeginForm() helper method:

- Html.BeginForm() allows you to mark the start of a form.
- Html.BeginForm() co-ordinates with the routing engine to generate a URL.
- Html.BeginForm() is responsible for producing the opening tag.

The general syntax of the `Html.BeginForm()` helper method is as follows:

```
@{Html.BeginForm(<action_method>,<controller_name>);}
```

where,

- **action_method:** Is the name of the action method.
- **controller_name:** Is the name of the controller class.

Once you use the **`Html.BeginForm()`** helper method to start a form, you need to end a form using the **`Html.EndForm()`** helper method.

Following code snippet shows using the `Html.BeginForm()` and `Html.EndForm()` helper methods:

```
@{Html.BeginForm("Browse","Home");}  
<p>Inside Form</p>  
@{Html.EndForm();}
```

In the above code the **`Html.BeginForm()`** method specifies the Browse action of the HomeController as the target action method to which the form data will be submitted.

You can also avoid using the **`Html.EndForm()`** helper method to explicitly close the form by using the **`@using`** statement before the **`Html.BeginForm()`** method.

`Html.Label()` helper method:

- Allows you to display a label in a form.
- Enables attaching information to other input elements, such as text inputs, and increase the accessibility of your application.

The general syntax of the `Html.Label()` helper method is as follows:

```
@Html.Label(<label_text>)
```

where,

label_text: Is the name of the label.

`Html.TextBox()` helper method:

- Allows you to display an input tag.
- Used to accept input from a user.

To create the html for a textbox with id = “fullName” and name = “fullName”

```
<input type="text" name = "fullName" id = "fullName">
```

OR

We can use the “TextBox” Html helper class method

```
@Html.TextBox("fullName")
```

There are several overloaded versions. To set a value, along with name of a textbox

```
@Html.TextBox("fullName","Adil")
```

To set HTML attributes, use the following overloaded version. Note that: We are passing HTML attributes (style and title) as an anonymous type.

Following code snippet shows using a Html.TextBox()method:

```
@{Html.BeginForm("Browse","Home");}
@Html.Label("User Name:")</br>
@Html.TextBox("textBox1")</br></br>
<input type="submit" value="Submit">
@{Html.EndForm();}
```

Html.TextArea() helper method:

- Allows you to display a element for multi-line text entry.</p>
- Enables you to specify the number of columns and rows to be displayed in order to control the size of the text area.

Following code snippet shows using a Html.TextArea() method:

```
@{Html.BeginForm("Browse","Home");}
@Html.Label("User Name:")</br>
@Html.TextBox("textBox1")</br></br>
@Html.Label(" Address:")</br>
@Html.TextArea("textare1")</br></br>
<input type="submit" value="Submit">
@{Html.EndForm();}
```

Html.Password() helper method:

You can use the **Html.Password()** helper method to display a **password** field with bulleted text inside in place of original password.

Following code snippet shows using a Html.Password() method:

```
@{Html.BeginForm("Browse","Home");}  
@Html.Label("User Name:")</br>  
@Html.TextBox("textBox1")</br></br>  
@Html.Label("Address:")</br> @Html.TextArea("textarea1")</br></br>  
@Html.Label("Password:")</br>@Html.Password("password")</br></br>  
<input type="submit" value="Submit">  
@{Html.EndForm();}
```

Html.CheckBox() helper method:

You can use the **Html.CheckBox()** helper method to display a check box that enables the user to select a true or false condition.

Following code snippet shows using a Html.CheckBox() method:

```
@{Html.BeginForm("Browse","Home");}  
@Html.Label("User Name:")</br>  
@Html.TextBox("textBox1")</br></br>  
@Html.Label("Address:")</br> @Html.TextArea("textarea1")</br></br>  
@Html.Label("Password:")</br>@Html.Password("password")</br></br>  
@Html.Label("I need updates on my mail:")  
@Html.CheckBox ("checkbox1")</br> </br>  
<input type="submit" value="Submit"> @ {Html.EndForm();}
```

In this code, the **Html.CheckBox()** helper method renders a hidden input in addition to the check box input. The hidden input ensures that a value will be submitted, even if the user does not select the check box.

Html.DropDownList() helper method:

- Return a **<select>** element that shows a list of possible options and also the current value for a field.
- Allows selection of a single item.

The general syntax of the Html.DropDownList() helper method is as follows:

where,

- **value1,value2** and **value3** are the options available in the drop-down list.
- **Choose:** Is the value at the top of the list.

Following code snippet shows using a `Html.DropDownList()` method:

In Above code, the **`Html.DropDownList()`** method creates a drop-down list in a form with `myList` as its name and contains three values that a user can select from the drop-down list.

`Html.RadioButton()` helper method:

The **`Html.RadioButton()`** helper method allows you to provide a range of possible options for a single value.

The general syntax of the `Html.RadioButton()` helper method is as follows:

```
@Html.RadioButton("name","value",isChecked)
```

where,

- **name:** Is the name of the radio button input element.
- **value:** Is the value associated with a particular radio button option.
- **isChecked:** Is a Boolean value that indicates whether the radio button option is selected or not.

```
@{Html.BeginForm("Browse","Home");}  
@Html.Label("User Name:")</br>  
@Html.TextBox("textBox1")</br></br>  
@Html.Label("Address:")</br> @Html.TextArea("textarea1")</br></br>  
@Html.Label("Password:")</br>@Html.Password("password")</br></br>  
@Html.Label("I need updates on my mail:")@Html.CheckBox("checkbox1")</br> </br>  
@Html.Label("Select your city:") @Html.DropDownList("myList", new SelectList(new  
[] { "New York", "Philadelphia", "California"}), "Choose")</> </br></br>  
Male @Html.RadioButton("Gender", "Male", true)</br>  
Female @Html.RadioButton("Gender", "Female")</br> </br>  
<input type="submit" value="Submit">  
@{Html.EndForm();}
```

In Above code, the `Html.RadioButton()` helper methods is used to create two radio buttons to accept the gender of a user.

`Url.Action()` helper method:

The **Url.Action()** helper method generates a URL that targets a specified action method of a controller.

The general syntax of the Url.Action() helper method is as follows:

```
@Url.Action(<action_name>, <controller_name>)
```

where,

- **action_name:** Is the name of the action method.
- **controller_name:** Is the name of the controller class.

Following code snippet shows the Url.Action() method:

```
<a href='@Url.Action("Browse", "Home")'>Browse</a>
```

Above code creates a hyperlink that targets the URL generated using the **Url.Action()** method. When a user clicks the hyperlink, the Browse action of the Home controller will be invoked.

Is it compulsory to use html helpers in ASP.Net MVC Application?

No, you can type the required html, but using html helpers will greatly reduce the amount of HTML markup that we have to write in a view. Views should be as simple as possible.

```
9      <br /><br /><br />
10
11
12      @{
13          Html.BeginForm("Index", "Home");
14      }
15
16      @Html.Label("Email", "Email : ") &nbsp;
17
18      @Html.TextBox("fullname", "Muhammad Farhan", new {
19          style = "background-color: red; color: green;",
20      })
21
22
23      <br />
24      <div class="form-group">
25
26          @Html.Label("pass", "Password : ") &nbsp;
27
28          @Html.Password("fullname", "Muhammad Farhan", new { @class = "form-control" })
29      </div>
30
31      @Html.DropDownList("Country List", new SelectList(new[] { "Pakistan", "india", "Afghanistan" }, "Select Country"))
32
33
34      @{
35          Html.EndForm();
36      }
```

Tag Helpers:

What is Tag Helpers?

- Tag helpers are used to render server side code on a Razor (.cshtml) file to create and render HTML elements.

Example:

- Create Image tag
- Create form tag
- Create Link etc.

Main Point:

- There are many built-in Tag Helpers for common tasks, such as creating forms, links, loading assets etc. Tag Helpers are authored in C#, and they target HTML elements based on the element name, the attribute name, or the parent tag.
- If you are familiar with HTML Helpers, Tag Helpers reduce the explicit transitions between HTML and C# in Razor views.

How to use of start tag helpers?

First set the scope

- @addTagHelper directive
- @removeTagHelper directive

Add the Directive in ViewImport File.

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

TAG HELPERS ADVANTAGES

Now, we need to understand why Tag Helpers is important or what are the advantages of it's over the HTML Helper objects. So that can compare these two different helper objects. So, before going to in deep discussion about Tag Helpers, let's discuss the main advantages of Tag Helpers over HTML Helpers objects

1. Tag Helpers use server-side binding without any server-side code.
2. This helper object is very much use full when HTML developers do the UI designing who does not have any idea or concept about Razor syntax.
3. It provides us an experience of working on basic HTML environments.the

4. It Supports rich IntelliSense environment support to create a markup between HTML and Razor

Anchor Tag Helper:

- Anchor tag helper is used to create a link `<a></a>` html element.
- By default all tag helpers attributes are prefixed with **asp-**

Main Attributes:

- Asp-controller
- Asp-action
- Asp-area
- Asp-route

Example:

```
<a asp-controller="Home" asp-action="Privacy">To go Privacy</a>
```

Example with Params:

```
<a asp-controller="Home" asp-action="getproductinfo" asp-route-id="@item.proid" >
```

Image Tag Helper:

- Image tag helper is used to give some additional capabilities to HTML img tag.
- Image tag helper is used to provide cache-busting behavior for static image.

Attribute:

- Asp-append-version

Example:

```

```

Environment Tag Helpers:

This Tag Helper renders the enclosed HTML content based on the current hosting environment conditionally. This helper object contains a single attribute named `name`. This attribute accepts string type value and value can be assigned as a comma separation method.

Attributes:

- Names
- Include
- Exclude

Example:

```
<environment names="Development">
```

```
  <h1>Development</h1>
```

```
</environment>
```

```
<environment names="Production">
```

```
  <h1>Production</h1>
```

```
</environment>
```

Another Example:

```
<environment include="Staging,Production">
```

```
  <strong>IWebHostEnvironment.EnvironmentName is Staging or Production</strong>
```

```
</environment>
```

```
<environment names="Production">
```

```
  <h1>Production</h1>
```

```
</environment>
```

```
<environment exclude="Development">
```

```
  <strong>IWebHostEnvironment.EnvironmentName is not Development</strong>
```

```
</environment>
```

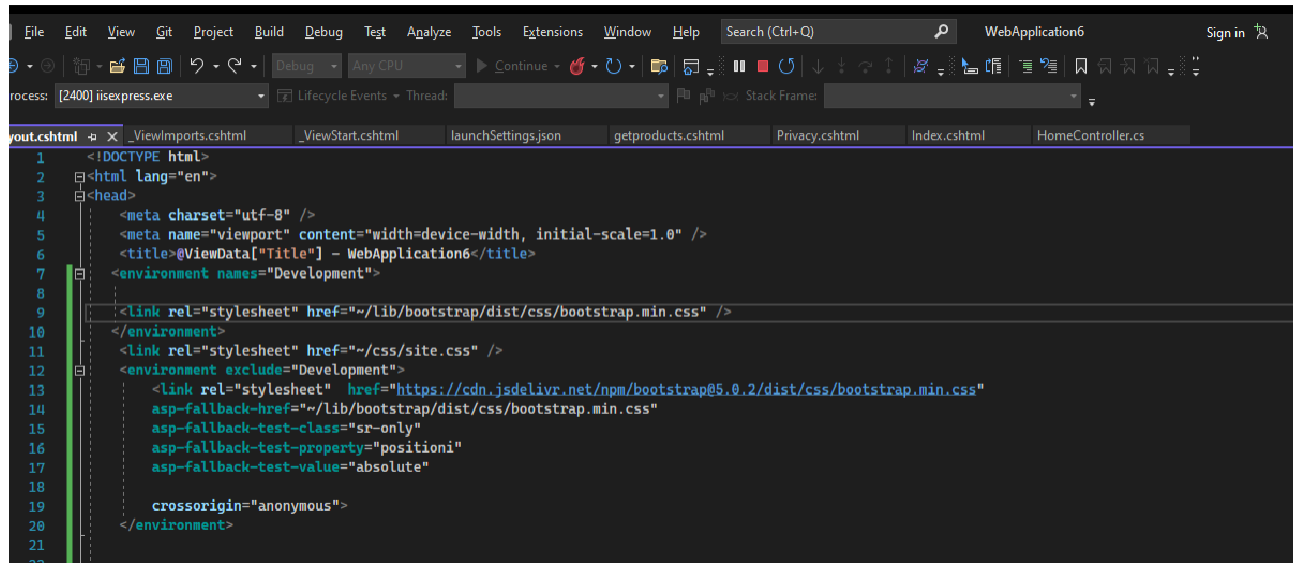
Link Tag Helper:

The Link Tag Helper generates a link to a primary or fall back CSS file. Typically the primary CSS file is on a Content Delivery Network (CDN).

A CDN:

- Provides several performance advantages vs hosting the asset with the web app.
- Should not be relied on as the only source for the asset. CDNs are not always available, therefore a reliable fallback should be used. Typically the fallback is the site hosting the web app.

The Link Tag Helper allows you to specify a CDN for the CSS file and a fallback when the CDN is not available. The Link Tag Helper provides the performance advantage of a CDN with the robustness of local hosting.

A screenshot of the Visual Studio Code editor interface. The top menu bar includes File, Edit, View, Git, Project, Build, Debug, Test, Analyze, Tools, Extensions, Window, Help, and a search bar. The status bar at the bottom shows 'process: [2400] iisexpress.exe' and 'Lifecycle Events - Thread:'. The editor window displays a file named 'out.cshtml' with the following code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>@ViewData["Title"] - WebApplication6</title>
7   <environment names="Development">
8     <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
9   </environment>
10  <link rel="stylesheet" href="~/css/site.css" />
11  <environment exclude="Development">
12    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
13      asp-fallback-href="~/lib/bootstrap/dist/css/bootstrap.min.css"
14      asp-fallback-test-class="sr-only"
15      asp-fallback-test-property="position"
16      asp-fallback-test-value="absolute"
17      crossorigin="anonymous">
18  </environment>
19 </head>
20 <body>
21
22
```

Create Form in Asp.Net Core using tag helpers:

```
<div class="container">
  <div class="row">

    <div class="offset-md-3 col-md-6">

      <form method="post" asp-action="AddProduct">

        <div class="form-group">

          <label asp-for="protitle">Title : </label>
          <input asp-for="protitle" class="form-control"/>
        </div>

        <input type="file" asp-for="proimg" />
        <br />
        <div class="form-group">

          <label asp-for="proshortdesc">Description (Short) : </label>
          <input asp-for="proshortdesc" class="form-control" />
        </div>
        <div class="form-group">
          <label asp-for="prolongdesc">Description (Long) : </label>
          <textarea asp-for="prolongdesc" class="form-control">
          </textarea>
        </div>
        <br />
        <div class="form-group">

          <label asp-for="proprice">Price : </label>
          <input asp-for="proprice" class="form-control" />
        </div>
      </form>
    </div>
  </div>
</div>
```

```
</div>
```

```
<button type="submit" value="addproduct" class="btn btn-primary" > Add product</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

Controller Work:

```
public IActionResult AddProduct()
{
    return View();
}
[HttpPost]
public JsonResult AddProduct(Product product)
{
    return Json(product);
}
```

MVC IN DETAIL WITH PRACTICAL

Model:

```
public IFormFile covering { get; set; }
```

View:

```
<label asp-for="covering"></label>  
<input asp-for="covering" /><br/>
```

Controller:

```
private readonly IWebHostEnvironment _webHostEnvironment;  
  
public HomeController(IWebHostEnvironment webHostEnvironment)  
{  
    _webHostEnvironment = webHostEnvironment;  
}
```

IN Action Method

```
if (model.img != null)  
{  
    folder = "images/";  
    folder += Guid.NewGuid().ToString() + "_" + model.img.FileName;  
    string serverfolder = Path.Combine(environment.WebRootPath, folder);  
  
    model.img.CopyTo(new FileStream(serverfolder, FileMode.Create));  
}
```

MVC IN DETAIL WITH PRACTICAL

Data Validation in Dotnet Core:

Validation is a process of checking an activity whether it meets the desired level of compliance.

In Asp.Net Core, validation is the process of checking fields of a form in order to meet the defined criteria.

Namespace: System.ComponentModel.DataAnnotations

How to check the Validation : ModelState.IsValid

When we developed any type of application such as web-based or desktop based, in both the application one is the key part of development is Data Validation or Form Validation. These validations always ensure us that user fills the correct form of data into the application so that we can process that data and save for future use. Actually, normally users always fill the form with necessary information and submit the form. But, sometimes the user make some mistakes. In this case, form validation is required to ensure that the user always provides the required information in the proper format which is required for the successful submission of the form. In this article, we will discuss this different types of form validations.

Server Side Validation:

In case of Server-side validation, information filled by the users are send to the server and then validate that information using the server-side language i.e. in the Asp.Net Core. In this process, if the data not validated, then a response along with proper error message is sent back to the client where these messages are shown to the user so that those wrong data can be rectified. This type of validation is always secure because if the user disabled the scripting language in the browser then also it will work.

Example:

```
public class Employee
{
    public int Id { get; set; }

    [Required]
```

```

[StringLength(100)]
public string EmployeeName { get; set; }

[Required]
[DataType(DataType.Date)]
public DateTime JoinDate { get; set; }

[Required]
[StringLength(100)]
public string Designation { get; set; }

[Range(0, 999.99)]
public decimal Salary { get; set; }

[Required]
public Gender Gender { get; set; }

public string Address { get; set; }
}

```

Data Validation	Definition
Required	This validation attributes makes any property as required or mandatory.
StringLength	This validation attributes validate any string property along with itslength
Compare	This validation attribute basically compares the two property values in a model match
EmailAddress	This validation attribute validates the email address format
Phone	This validation attribute validates the phone no format
CreditCard	This attribute validates a credit card format
Range	This validation attributes any property to check it is exist within the given range or not.

Url	This validation attributes validates the property contains an URL format or not
RegularExpression	This validation attributes normally match the data which specified the regular expression format.

Example:

View: (HTML):

```
<div class="text-danger" asp-validation-summary="All">
</div>

<form asp-controller="Home" asp-action="Index" method="post">
  <label asp-for="Name"></label>
  <input asp-for="Name" /><br />
  <span asp-validation-for="Name" class="text-danger"></span>
  <br/>
  <label asp-for="Email"></label>
  <input asp-for="Email" /><br />
  <span asp-validation-for="Email" class="text-danger"></span>
  <br/>
  <label asp-for="password"></label>
  <input asp-for="password" /><br />
  <span asp-validation-for="password" class="text-danger"></span>
  <br/>
  <label asp-for="conpassword"></label>
  <input asp-for="conpassword" /><br />
  <span asp-validation-for="conpassword" class="text-danger"></span>
  <br/>
  <label asp-for="channel"></label>
  <input asp-for="channel" /><br />
  <span asp-validation-for="channel" class="text-danger"></span>
  <br/>
  <label asp-for="Age"></label>
  <input asp-for="Age" /><br />
  <span asp-validation-for="Age" class="text-danger" ></span>
  <br/>
  <input type="submit" value="Submit" class="btn btn-primary"/>
</form>
```

Model:

```
public class Employee
{
    [Required(ErrorMessage = "Name is Must Fill")]
    [StringLength(15,MinimumLength = 3,ErrorMessage = "Name length must be 3")]
    public string Name { get; set; }

    [Required]
    [EmailAddress(ErrorMessage = "Email is Must")]
    public string Email { get; set; }
}
```

```

[Required(ErrorMessage = "Age is Must !")]
[Range(10,50,ErrorMessage = "Must be between 10 to 50")]
public int? Age { get; set; }
[Required]
[RegularExpression(@"(?=\d)(?=[a-z])(?=[A-Z])(?=.*[@#$%^&+=]).*$",ErrorMessage =
"Password must be strong!")]
public string password { get; set; }
[Required]
[Compare("password")]
[Display(Name = "Confirm Password")]
public string conpassword { get; set; }
[Required]
[Url()]
public string channel { get; set; }

}

```

Controller:

```

public IActionResult Index()
{
    return View();
}
[HttpPost]
public IActionResult Index(Employee e)
{
    return View();
}

```

ENTITY FRAMEWORK:

Install 3 packages:

Microsoft.entityframeworkcore.sqlserver

Microsoft.entityframeworkcore.Tools

Microsoft.entityframeworkcore.design

Step1: Install Entity framework Core from manage nuget package.

Step2: Create Model class database Context Class.

Step3: create database connection in appsetting.json file

Step4: Register connection in Program.cs

Step4: Create Controller with Views.

Step5: Create Migration by using this command “add-migrationInitialCreate”.

Step6: Then type “update-database” command.

Explain:

Step1: is so simple.

Step2:

```
using Microsoft.EntityFrameworkCore;

namespace modelwork.Models
{
    public class StudentContext : DbContext
    {
        public StudentContext(DbContextOptions option) : base(option)
        {

        }

        public DbSet<Student> Students { get; set; } } }
```

Step3:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
```

```

    "Microsoft.AspNetCore": "Warning"
  }
},
"ConnectionStrings": {
  "dbcs": "server=MUHAMMAD-FARHAN; database=schoolsys; trusted_connection=true; TrustServerCertificate=true;";
},
"AllowedHosts": "*"
}

```

Step4:

```

var provider = builder.Services.BuildServiceProvider();

var config = provider.GetRequiredService<IConfiguration>();

builder.Services.AddDbContext<StudentDbContext>(item =>
item.SqlServer(config.GetConnectionString("dbcs"));

```

Step5:

Add-Migration init

Command: add-migration createdb

Step6:

Update-database

Step7:

Create Controller:

CRUD using Simple steps:

Index show list data:

```

private readonly StudentDbContext studentDbContext;

public HomeController(StudentDbContext studentDbContext)
{
    this.studentDbContext = studentDbContext;
}

public IActionResult Index()
{

```

```

        var students = studentDbContext.Students.ToList();

        return View(student);
    }

```

Create Student:

```

[HttpPost]
public IActionResult Create(Student student)
{
    if (ModelState.IsValid)
    {
        studentDbContext.Students.add(student);
        studentDbContext.SaveChanges();
        return RedirectToAction("Index","Home");
    }
    return View(student);
}

```

Details of Students:

```

public IActionResult Details(int id)
{
    var student = studentDbContext.Students. FirstOrDefault (x => x.id = id);
    return View(student);
}

```

Async Methods add in Action Methods:

```

public async Task<IActionResult> Index()
{
    var students = await studentDbContext.Students.ToListAsync();
    return View(students);
}

```

Relation CRUD: Departments & Employees

Models:

```

Department.cs
public class Department

```

```
{
public int Id { get; set; }
public string Name { get; set; }

public List<Employee> Employees { get; set; }
}
```

Employee.cs

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Salary { get; set; }

    // FK
    public int DepartmentId { get; set; }

    public Department Department { get; set; }
}
```

New Migrations:

Command: Add-Migration empdepartment

Command: Update-Database

Employee Controller:

Using

```
using Microsoft.AspNetCore.Mvc.Rendering;
```

```
public IActionResult Index()
{
    var employees = _context.Employees.Include(e => e.Department);
    return View(employees.ToListAsync());
}
```

```
public IActionResult Details(int? id)
```

```
var employee = await _context.Employees
```

```

        .Include(e => e.Department)
        .FirstOrDefaultAsync(m => m.EmployeeId == id);
    if (employee == null)
    {
        return NotFound();
    }
    return View(employee);
}
public IActionResult Create()
{
    ViewBag.DepartmentId = new SelectList(_context.Departments, "Id", "Name");
    return View();
}

[HttpPost]
public async Task<IActionResult> Create(Employee emp)
{
    _context.Add(emp);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

public async Task<IActionResult> Edit(int id)
{
    var emp = await _context.Employees.FindAsync(id);
    ViewBag.DepartmentId = new SelectList(_context.Departments, "Id", "Name",
    emp.DepartmentId);
    return View(emp);
}

```

View:

index.html from employee:

```

<td>
    @if (item.Department != null)
    {
        @Html.DisplayFor(modelItem => item.Department.Name)
    }
</td>

```

Database First Approach

Steps:

1. Install Entity Framework
2. Scaffold-DbContext
`'server=ServerName;database=DbName;trusted_connection=true'`
Microsoft.Entity.FrameworkCore.SqlServer-OutputDir Models
3. Add Connection string in startup file.

```
Var connection = "";
```

```
Services.AddDbContext<StudentDbContext>(options=>  
options.UseSqlServer(connection));
```

4. Add MVC Controller with Views.

How to Update Model Classes.

Scaffold-DbContext

“server=ServerName;database=DbName;trusted_connection=true”

Microsoft.Entity.FrameworkCore.SqlServer-OutputDir Models -Force

Creating Dropdown list:

```
<div class="form-group">
```

```
    @Html.DropDownList("Gender", Html.GetEnumSelectList<Gender>(), "Select Gender", new { @class = "form-control" })
```

```
</div>
```


Identity Setup:

- 1) Download identity Framework.
- 2) Change DbContext to IdentityDbContext.
- 3) Service Add:

```
services.AddIdentity<IdentityUser,  
IdentityRole>().AddEntityFrameworkStores<bazarContext>();
```

- 4) Add-migration

- 5) If Error in Migration so Add in DB-Context

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    base.OnModelCreating(modelBuilder);  
    modelBuilder.Seed();  
}
```

- 6)

Sign UP Work

Sign Up Model:

```
public class SignupModel  
{  
  
    [Required]  
    [EmailAddress]  
    public string Email{ get; set; }  
    [Required]  
    [DataType(DataType.password)]  
    public string password{ get; set; }  
    [Required]  
    [DataType(DataType.password)]  
    [Compare("password")]  
    public string Confirmpassword { get; set; }  
}
```

Controller:

```
private readonly UserManager<IdentityUser> userManager;  
private readonly SignInManager<IdentityUser> signInManager;  
  
public AccountController(UserManager<IdentityUser> userManager,  
SignInManager<IdentityUser> signInManager)  
{
```

```

        this.userManager = userManager;
        this.signInManager = signInManager;
    }

    public IActionResult Register()
    {
        return View();
    }
    [HttpPost]
    public async Task<IActionResult> Register(SignupModel model)
    {
        if (ModelState.IsValid)
        {
            var identity = new IdentityUser()
            {
                Email = model.Email,
                UserName = model.Email
            };

            var result = await userManager.CreateAsync(identity,
model.Password);

            if (result.Succeeded)
            {
                await signInManager.SignInAsync(identity, isPersistent : false);
                return RedirectToAction("Index", "Home");
            }

            foreach (var error in result.Errors)
            {
                ModelState.AddModelError(string.Empty, error.Description);
            }
        }

        return View();
    }

```

Logout:

```

    public IActionResult Logout(){
        signInManager.SignOutAsync();

        return RedirectToAction("Index", "Home");
    }

```

Login Work:

Login Model:

```

public class LoginModel
{
    [Required]
    [EmailAddress]
    public string Email{ get; set; }
    [Required]
    [DataType(DataType.password)]
    public string password { get; set; }
}

```

Controller:

```

[HttpGet]
public IActionResult Login()
{
    return View();
}
[HttpPost]
public async Task<IActionResult> Login(LoginModel model)
{
    if (ModelState.IsValid)
    {
        var result = await signInManager.PasswordSignInAsync(model.Email,
model.password,false,false);

        if (result.Succeeded)
        {
            return RedirectToAction("Index", "Home");
        }

        ModelState.AddModelError("", "Invalid Credentials !");
    }

    return View(model);
}

```

Creating Roles:

Steps:

- 1) Create Controller
- 2) Create Model CreateRoleModel
- 3) Add RoleManager<IdentityRole> in Constructor
- 4) Create Role Method and View in Controller
- 5) Create Post Role Method

```

public IActionResult Create()
{
    return View();
}

[HttpPost]

public async Task<IActionResult> Create(CreatingRoleModel model)
{
    if (ModelState.IsValid)
    {
        IdentityRole user = new IdentityRole()
        {
            Name = model.RoleName
        };

        IdentityResult result = await roleManager.CreateAsync(user);

        if (result.Succeeded)
        {
            return RedirectToAction("RolesList", "Administrator");
        }

        foreach (IdentityError error in result.Errors)
        {
            ModelState.AddModelError("", error.Description);
        }
    }
    return View(model);
}

```

6) View Roles List Action Method:

```

[HttpGet]
public IActionResult RolesList()
{
    var roles = roleManager.Roles;

    return View(roles);
}

```

7) View Roles List

```

<div class="container mt-3">
    <div class="row">

        @foreach (var role in Model)
        {

            <div class="col-md-12">

```

```
<p class="bg-light">Role Id : @role.Id</p>
<p>Role Name : @role.Name</p>
<a class="btn btn-primary" asp-action="EditRole"
  asp-controller="Administrator" asp-route-
id="@role.Id">edit</a>
  <a class="btn btn-danger">Delete</a>
</div>
}
</div>
</div>
```

8) Delete Role:

Upload Single Image:

```
var folder = "";
    if (model.pimg != null)
    {
        folder = "images/";

        folder = await MuplImages(folder, model.pimg);
    }
```

Method :

```
public async Task<string> MuplImages(string folderpath, IFormFile
file)
    {

        folderpath += Guid.NewGuid().ToString() + "_" +
file.FileName;

        string serverfolder =
flath.Combine(_webHostEnvironment.WebRootPath, folderpath);

        await file.CopyToAsync(new FileStream(serverfolder,
 FileMode.Create));

        return "/" + folderpath;
    }
```

Upload Multiple Image:

```
var gfolder = "";
if (model.GalleryImages != null)
{
    gfolder = "images/Gallery/";

    model.Gallery = new List<flproductfictures>();

    foreach (var file in model.GalleryImages)
    {
        var gallery = new flproductfictures()
        {
            Imgsrc = await MuplImages(gfolder, file),
            flproductid = id
        };

        model.Gallery.Add(gallery);

        await _context.SaveChangesAsync();
    }
}

public async Task<string> MuplImages(string folderpath, IFormFile file)
{
    folderpath += Guid.NewGuid().ToString() + "_" + file.FileName;

    string serverfolder = flath.Combine(_webHostEnvironment.WebRootflath,
folderpath);

    await file.CopyToAsync(new FileStream(serverfolder, FileMode.Create));

    return "/" + folderpath;
}
```